

is to alter temperature or top_t but not both.

Max length: By changing the max length, you can control how many tokens the model produces, thereby avoiding lengthy or irrelevant responses and controlling costs.

Stop sequence: Specifying stop sequences is another way to control the length and structure of the model's response.

Frequency penalty: The next token is subject to a penalty known as the frequency penalty. The likelihood that a term will reappear decreases with increasing punishment. This configuration reduces word repetition in the model's responses.

Presence penalty: Similar to frequency penalty, the presence penalty is a constant penalty for repeated tokens, preventing excessive repetition. It can be increased for innovative or diversified text, or reduced for maintaining focus. The penalty is equal for tokens appearing twice or ten times.

As with top_p and temperature, it is generally advised to change the presence penalty or frequency but not both. Remember that the version of LLM you use will affect your results.

Prompt engineering basics

Prompt engineering is essentially the art of creating efficient prompts or inputs to direct AI language models towards intended results. It focuses on creating and refining prompts to effectively apply and construct LLMs for a range of use cases and applications (Figure 4).

Prompt engineering skills help to better understand the capabilities and limitations of LLMs. Researchers use it to improve safety and the capacity of LLMs on a wide range of common and complex tasks such as question-answering and arithmetic reasoning. Prompt engineering is a tool used by developers to create reliable and efficient prompting methods that communicate with LLMs and other tools.

Elements of a prompt

In prompt engineering, various elements play a critical role in shaping the user's prompt and obtaining the desired output. Let's explain each of these.

Role: The role defines the particular function or identity that the model is anticipated to carry out during the conversation. It establishes the tone and degree of skill anticipated from the model's answers.

Instruction: A particular task or directive that you would like the model to follow.

Context: Extraneous data or outside context that can direct the model towards more appropriate answers.

Input data: The information or query for which we are looking for a response.

Output indicator: Indicates the format or kind of output.

Constraint: The guidance should support readability and efficiency in a large-scale project environment while also improving the code's performance and cleanliness.

Figure 5 shows examples of the different elements of a prompt.

Prompting techniques

Let's now take a quick look at the more sophisticated prompt engineering techniques that help accomplish complicated tasks, and enhance LLM performance and reliability.

Zero-shot prompting: Zero-shot prompting refers to giving the model a prompt that is not included in the training set but allows it to provide the desired outcome.

Example

User prompt:

Classify the text into positive, neutral or negative. Text: That idea was awesome.

Few-shot prompting:

LLMs perform poorly on more difficult tasks when operating in the zero-shot mode. Few-shot prompting can help the model perform better by guiding it using demonstrations provided in the prompt. The demonstrations are used to condition the model for later instances in which we want it to produce a response.

Example

User prompt:

Classify the text into positive, neutral, or negative.

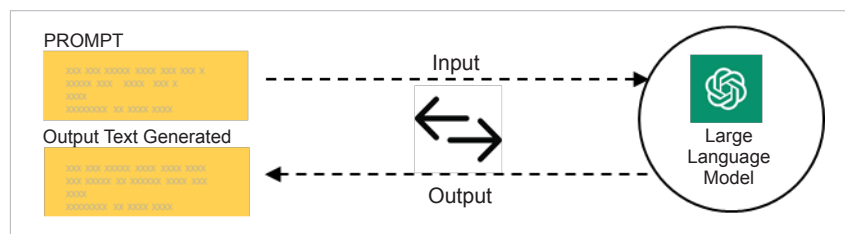


Figure 4: Prompt engineering

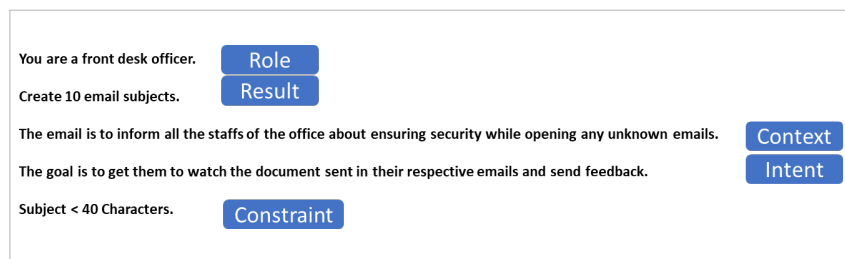


Figure 5: Different elements of a prompt